



SPORTS DEPARTMENT DATA MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KUMUTHA R (950522104023)

PAVITHRA S (950522104034)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

**Dr. SIVANTHI ADITANAR COLLEGE OF ENGINEERING,
TIRUCHENDUR**

ANNA UNIVERSITY :: CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**SPORTS DEPARTMENT DATA MANAGEMENT SYSTEM**” is the bonafide work of “**KUMUTHA R (950522104023), PAVITHRA S (950522104034)**” who carried out the project work under my supervision.

SIGNATURE

**Dr.G.WISELIN JIJL,M.E.,Ph.D.,
PRINCIPAL AND
HEAD OF THE DEPARTMENT**

Department of Computer Science
and Engineering.

Dr. Sivanthi Aditanar College of
Engineering.

Tiruchendur-628215.

SIGNATURE

**Dr.D.KESAVARAJA,M.E.,Ph.D.,
SUPERVISOR AND
PROFESSOR**

Department of Computer Science
and Engineering.

Dr. Sivanthi Aditanar College of
Engineering.

Tiruchendur-628215.

Submitted for B.E., Degree viva-voce examination held at **Dr. Sivanthi Aditanar College of Engineering** on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We take this opportunity to thank our honourable founder **Padma shri Dr.B.Sivanthi Adityan** and our beloved chairman **Sri.S.Balasubramanian Adityan**, for providing us with an excellent infrastructure and conducive atmosphere for developing the project.

We sincerely thank our respected Principal and Head of the Department of Computer Science and Engineering (CSE), **Dr.G.Wiselin Jiji, M.E., Ph.D.**, for giving us the opportunity to showcase our professional skills through this project. Her valuable guidance and continuous support have been instrumental in enhancing our technical and academic growth.

We initiate our genuine and heartfelt thanks and credits to our project guide **Dr.D.Kesavaraja M.E., Ph.D.** Professor, Department of Computer Science and Engineering, for his excellence in guidance and fullest support in successful development of this project in a contented way.

We thank all our teaching and non-teaching staff members of the Computer Science Department for their passionate support, for helping us to identify our mistakes and also for the appreciation they gave us in achieving our goal.

ABSTRACT

This framework presents a comprehensive Sports Department Data Management System developed using the PERN stack (PostgreSQL, Express.js, React.js, Node.js). The system addresses the inefficiencies of manual sports administration by providing a centralized, role-based platform for four user types: Admin, Director, Staff, and Student. Secure authentication is implemented using bcrypt password hashing and JSON Web Tokens (JWT), ensuring that only authorized users can access the system. Role-based access control is strictly enforced at both frontend and backend levels, presenting each user with a customized dashboard that contains only the features relevant to their responsibilities. Administrators can manage users and monitor all activities; directors can create events and training schedules; staff can mark attendance, record performance, and manage kit issuance; students can browse events, enroll, and view their achievements.

The system was successfully tested for all core operations including event creation, enrollment tracking, attendance marking, performance recording, kit management, audit logging, selection reports, and student history summaries. PostgreSQL ensures data consistency across related tables, while the REST API design separates frontend and backend for easier maintenance. The framework eliminates manual record-keeping, reduces communication delays, and provides real-time data access. Minor performance bottlenecks observed during complex report generation can be resolved with future optimizations such as database indexing and caching. Overall, this framework is scalable, reusable, and suitable for university sports departments, athletic academies, and similar institutions requiring structured, role-based activity management.

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1.	INTRODUCTION	1
	1.1 Overview	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope of the Project	2
2.	LITERATURE SURVEY	3
3.	METHODOLOGY	4
	3.1 System Architecture	6
	3.2 Authentication and Authorization	6
	3.3 Role-Based Dashboards	6
	3.4 Database Schema	6
	3.5 API Design	7

	3.6 Frontend Design	7
	3.7 Backend Design	7
4.	RESULT AND DISCUSSION	8
	4.1 Results	8
	4.2 Discussion	17
5.	CONCLUSION	18
6.	FUTURE ENHANCEMENTS	20
	REFERENCES	22
	APPENDIX A-SOURCE CODE	24

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	User Authentication Interface	8
4.2	Admin - Dashboard Overview	9
4.3	System Audit Logs and history	9
4.4	Staff CRUD with Search and Status	10
4.5	Statistics Dashboard with event table	10
4.6	Recent Achievements and Rankings	11
4.7	Manages student profiles with statistics	11
4.8	Training programs with statistics cards	12
4.9	Displays student status list with checkboxes	12
4.10	Metric and rating fields for student performance	13
4.11	Selection report with attendance, rating and score	13
4.12	Kit stock and issue management	14
4.13	Student Profile and Performance Summary	14
4.14	Browse and register for sports events	15
4.15	View registered events with status	15
4.16	Training Programs with Session Details and Attendance	16
4.17	View issued kits and request return	16

LIST OF ABBREVIATIONS

SNO	ABBREVIATION	EXPANSION
1	API	Application Programming Interface
2	CSV	Comma Separated Values
3	CRUD	Create, Read, Update and Delete
4	DB	Database
5	JWT	JSON Web Token
6	PG	PostgreSQL
7	REST	Representational State Transfer
8	UI	User Interface
9	URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

Managing a sports department involves coordinating administrators, directors, staff, and students with distinct responsibilities. Traditional manual methods cause inefficiencies, data redundancy, and poor communication. This project develops a centralized, role-based management system for secure authentication, role-specific dashboards, event registration, attendance tracking, and performance monitoring. The system eliminates the need for scattered spreadsheets and paper records by providing a single platform where each user logs in and accesses only the features relevant to their role. This reduces administrative overhead, prevents data duplication, and ensures that all stakeholders stay informed about events, enrollments, attendance and achievements in real time, thereby bringing transparency and efficiency to sports department operations.

1.2 PROBLEM STATEMENT

Manual sports management is labor-intensive, error-prone, and lacks scalability. Traditional methods fail to provide role-based access control, leading to fragmented workflows and poor coordination among stakeholders. There is a lack of comprehensive systems that securely manage distinct user roles while supporting event creation, enrollment, attendance, and performance tracking. Ensuring data integrity, real-time access, and seamless role-specific functionality across varying user hierarchies remains a significant challenge.

1.3 OBJECTIVES

The main objective is to design a Sports Department Data Management System using the PERN stack to automate sports administration. The system provides role-based access for Admin, Director, Staff, and Student with customized dashboards. Key objectives include enabling administrators to manage users, directors to create events and schedules, staff to mark attendance and record performance, and students to browse events and view achievements. The system ensures secure authentication using JWT and real-time data management. A fully functional web-based prototype will be developed.

1.4 SCOPE OF THE PROJECT

The system provides secure login and role-specific dashboards supporting user management, event creation, enrollment tracking, attendance marking, performance recording, and achievement review. Applications include university sports departments and athletic academies. The system is scalable for future enhancements such as analytics dashboards, automated notifications, mobile integration, and reporting tools. With additional development, it can be extended to other domains requiring structured user hierarchies and activity management.

CHAPTER 2

LITERATURE SURVEY

Sports department management plays a vital role in educational institutions and athletic organizations by efficiently managing users, events, attendance and performance records. Traditional management methods, such as paper files and spreadsheets, were time-consuming, error-prone and difficult to maintain. They also lacked proper coordination among administrators, directors, staff and students.

The development of web-based management systems has improved sports administration by providing centralized and role-based access to data. Modern technologies like the PERN stack (PostgreSQL, Express.js, React.js, and Node.js) enable the creation of responsive and secure applications. React.js supports dynamic user interfaces, while Node.js and Express.js provide backend services and REST APIs. Security features such as JSON Web Tokens (JWT) and bcrypt ensure secure authentication and data protection.

Existing systems provide features like event scheduling and attendance tracking, but many still lack real-time updates, advanced performance monitoring, and effective role-based access control. Challenges such as manual data entry, adapting systems to different institutional needs, and maintaining data privacy continue to exist.

Overall, modern web technologies have significantly improved sports department management by making operations more organized, secure and efficient. However, further improvements are needed in analytics, real-time synchronization and user experience to meet the growing demands of modern institutions.

CHAPTER 3

METHODOLOGY

This framework takes user login credentials as input and authenticates them against the PostgreSQL database using secure password hashing with bcrypt. Upon successful authentication, this system retrieves the user's role Admin, Director, Staff, or Student from the database and generates a JSON Web Token (JWT) for session management. The role determines which dashboard and functionalities are accessible to the user.

Role-based functionalities include

Administrators can

- Appoint directors and physical staff
- Register and manage students
- Monitor all activities across the system

Directors can

- Create sports and events
- Develop training schedules
- Monitor staff activities
- Review achievements

Staff members can

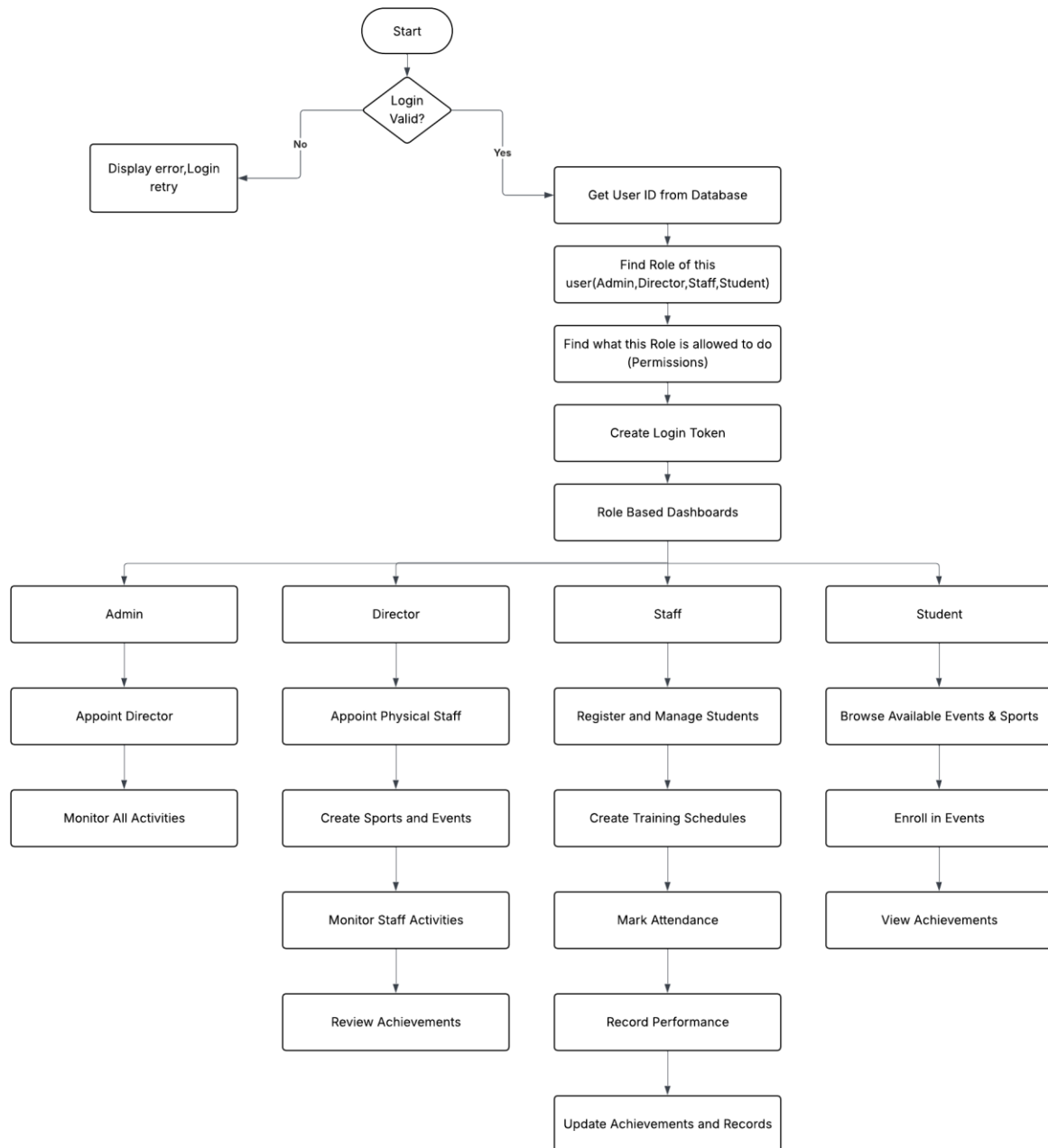
- Mark attendance
- Record performance
- Update achievements and records

Students can

- Browse available events and sports
- Enroll in activities
- View their achievements

The entire system is built using the PERN stack with PostgreSQL for data storage,

Express.js for backend API development, React.js for dynamic frontend dashboards and Node.js for server-side logic. The process is shown in the flowchart below:



3.1. SYSTEM ARCHITECTURE

The system starts with user login. Invalid credentials show an error and retry. On success, the system retrieves the user ID and role (Admin, Director, Staff, Student) from the database, then creates a login token. Based on the role, a specific dashboard is presented: Admin manages directors; Director creates events; Staff handles students, training, attendance and performance. Students can browse and enrolls in events.

3.2. AUTHENTICATION AND AUTHORIZATION

The system implements authentication using bcrypt for secure password hashing and JWT (JSON Web Token) for session management. Upon login, user credentials are verified against the PostgreSQL database. If valid, a token containing the user's role and ID is generated and stored locally. Authorization is enforced at every API request by validating the token and checking role-based permissions, ensuring that users can only access functionalities permitted.

3.3. ROLE-BASED DASHBOARDS

This system provides separate dashboards for Admin, Director, Staff and Student based on their role. The Admin dashboard includes user management and activity monitoring. The Director dashboard allows creating events, scheduling training, and reviewing achievements. The Staff dashboard supports attendance marking, performance recording, and updating records. The Student dashboard enables browsing events, enrolling and viewing personal achievements.

3.4. DATABASE SCHEMA

The PostgreSQL database contains tables for users, roles, events, enrollments, attendance, performance, and achievements. The users table stores id, name, email, hashed password and role. The events table includes event details such as

name, date, venue and schedule. The enrollments table links students to events, while attendance and performance tables record participation and scores.

3.5. API DESIGN

The backend exposes REST APIs for all operations. Authentication endpoints include POST /api/auth/login and POST /api/auth/logout. User management APIs support CRUD operations on users. Event APIs handle creation, listing, enrolment and deletion. Attendance and performance APIs allow marking attendance and recording scores. All APIs require a valid JWT token in the Authorization header.

3.6. FRONTEND DESIGN

The frontend is built with React.js, featuring a login page with role selection, a main dashboard that renders role-specific components, and separate modules for each function. State management is handled using React Context API and Axios is used for API calls. The interface is responsive and includes tables, forms, buttons, and search functionality for easy data management.

3.7. BACKEND DESIGN

The backend uses Node.js with Express.js, following an MVC pattern. Controllers handle business logic, models interact with the PostgreSQL database using raw SQL queries, and middleware manages authentication and role-based authorization. Environment variables store database credentials and JWT secret. The server runs on a specified port and listens for API requests.

CHAPTER 4

RESULTS AND DISCUSSION

The Sports Department Data Management System was successfully implemented using the PERN stack with four user roles: Admin, Director, Staff and Student. Role-based access control ensured secure, role-specific dashboards. Admins managed users, directors created events, staff marked attendance and recorded performance, and students browsed events and viewed achievements. The system provided real-time data management, secure authentication and efficient coordination, reducing manual effort and improving transparency.

4.1 RESULT

Fig: 4.1 The Login screen with role selection and enter username/ID with password and optionally enable "Remember this device".

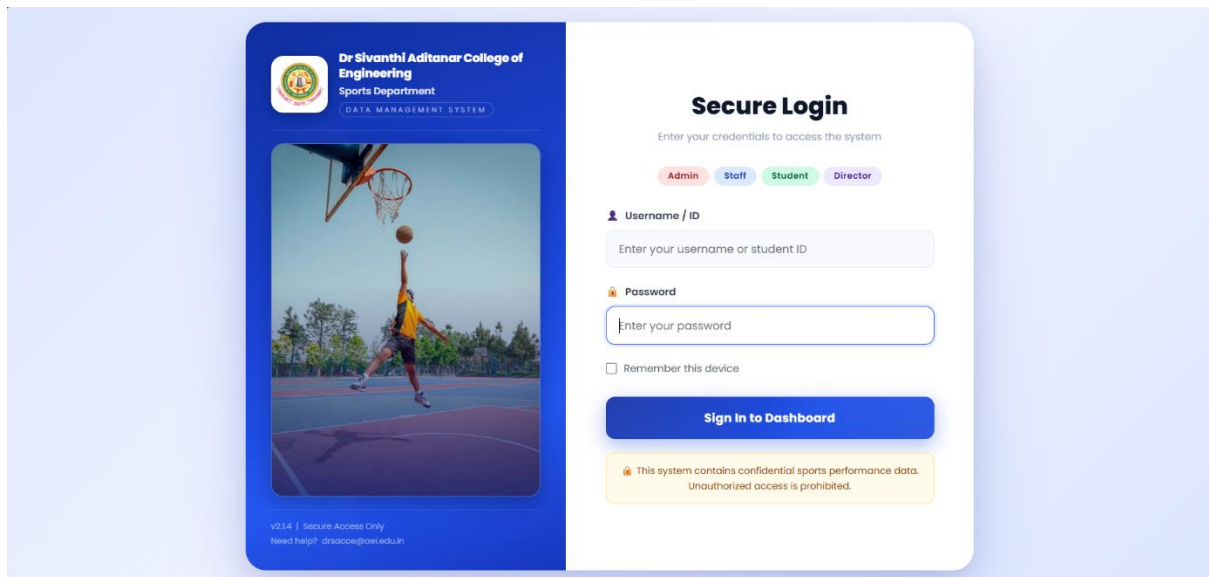


Fig: 4.1 User Authentication Interface

Fig: 4.2 The Admin dashboard: manage directors via table (search, edit/delete), with create director, switch role, and logout options.

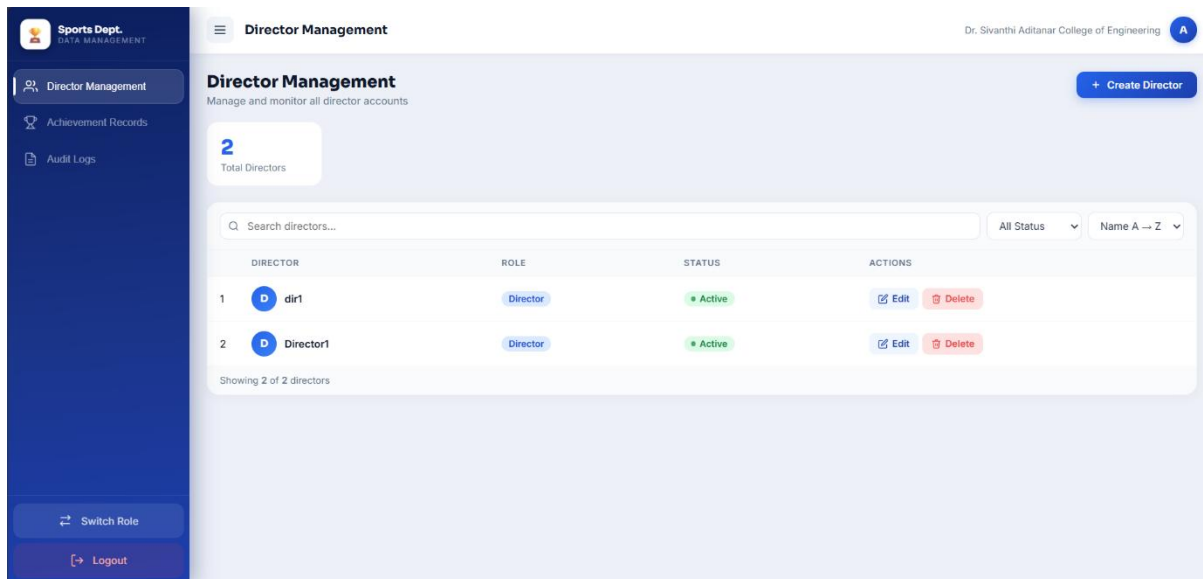


Fig: 4.2 Admin - Dashboard Overview

Fig: 4.3 The audit logs page displays total logs and role-wise counts. It shows action tables by role and date/time, with switch role and logout buttons.

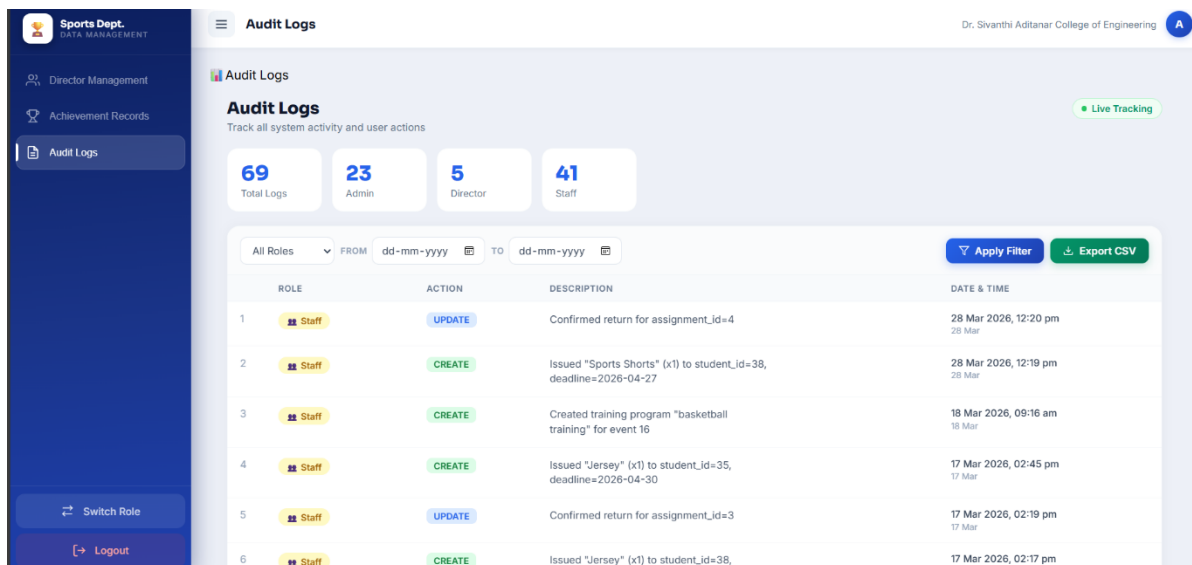


Fig: 4.3 System Audit Logs and history

Fig: 4.4 The Staff Management page displays total staff count with search, edit, delete and Create Staff button.

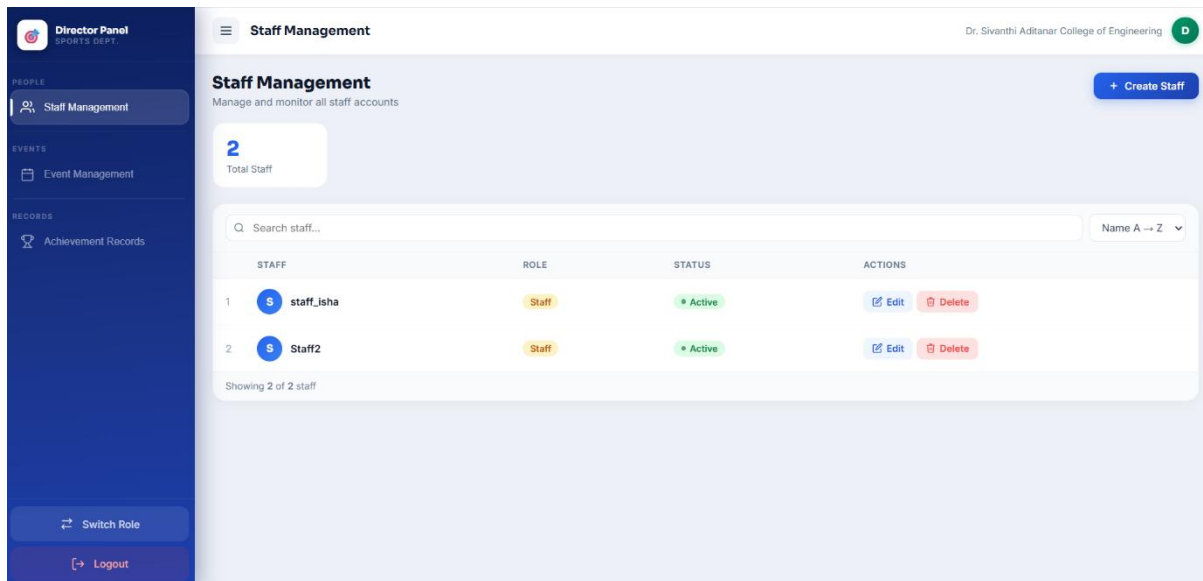


Fig: 4.4 Staff CRUD with Search and Status

Fig: 4.5 The Event Management page shows total events, type counts, search bar, and event details with edit and delete options.

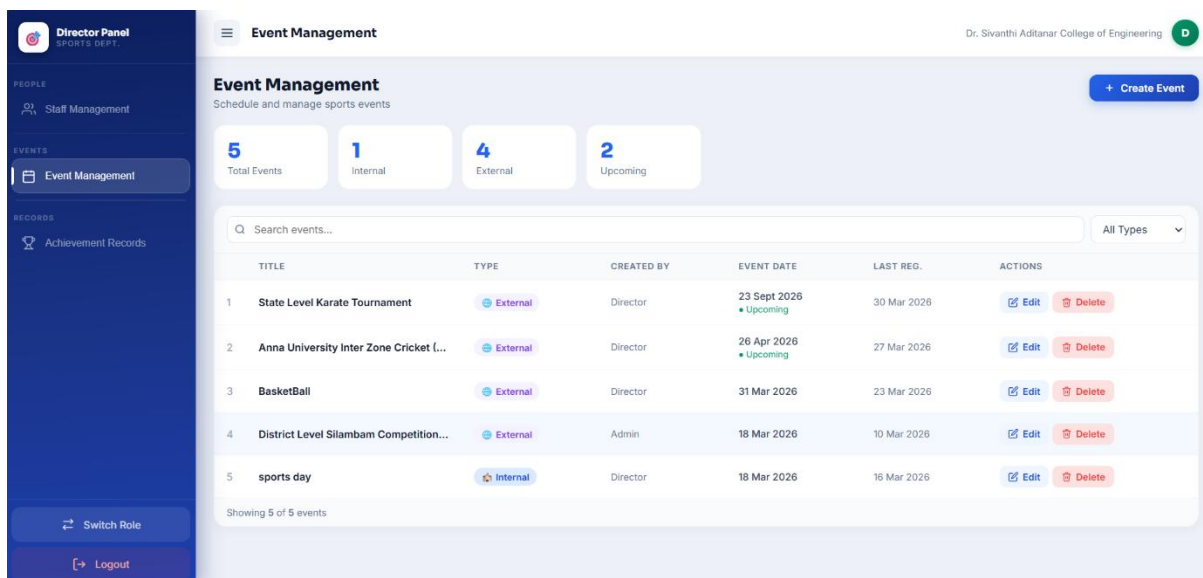


Fig: 4.5 Statistics Dashboard with Event Table

Fig: 4.6 The Achievements page displays medal counts, department leaderboard, top sports and a list of recent achievements with student names and event details.

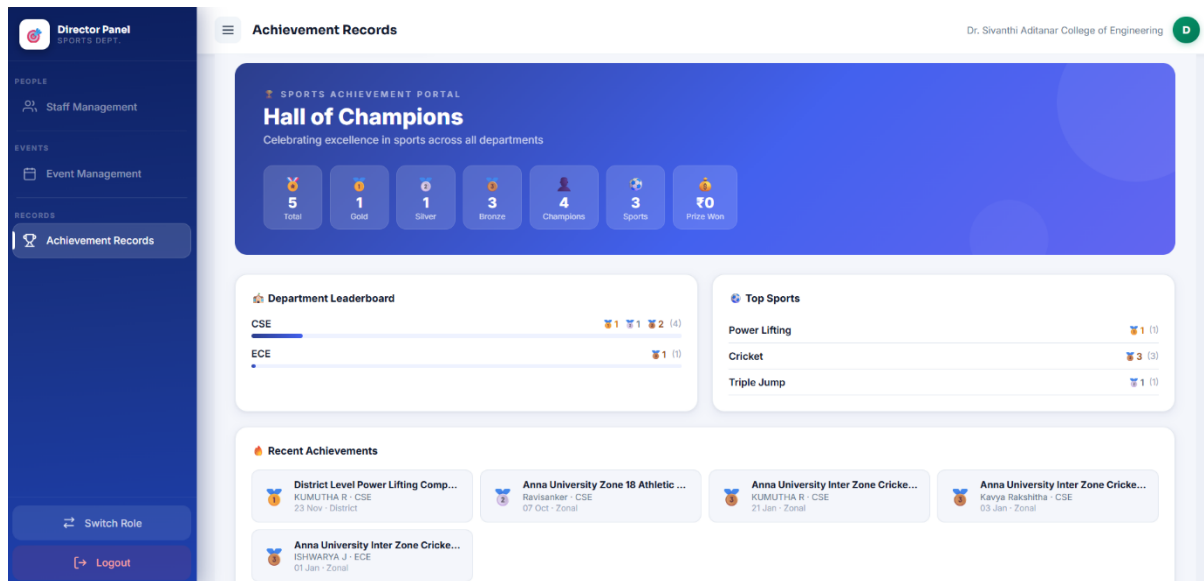


Fig 4.6 Recent Achievements and Rankings

Fig: 4.7 The Student Management page displays student statistics and a search bar and a table with student details and view/edit/delete actions.

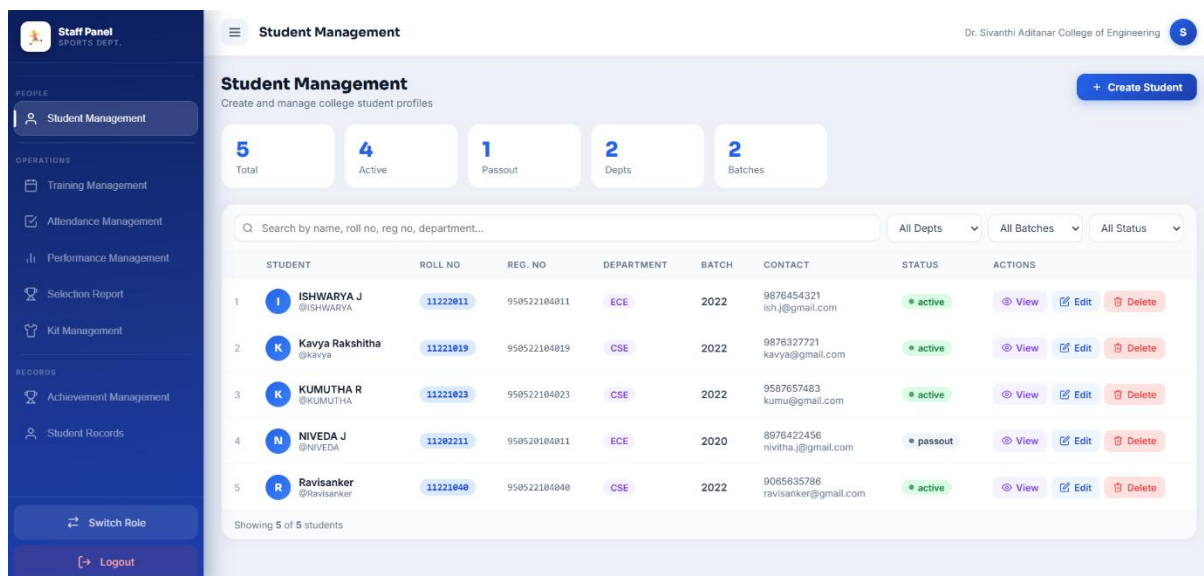


Fig: 4.7 Manages student profiles with statistics

Fig: 4.8 The Training Management page displays statistics cards for programs, active, completed, students and sessions, with a list of training programs.

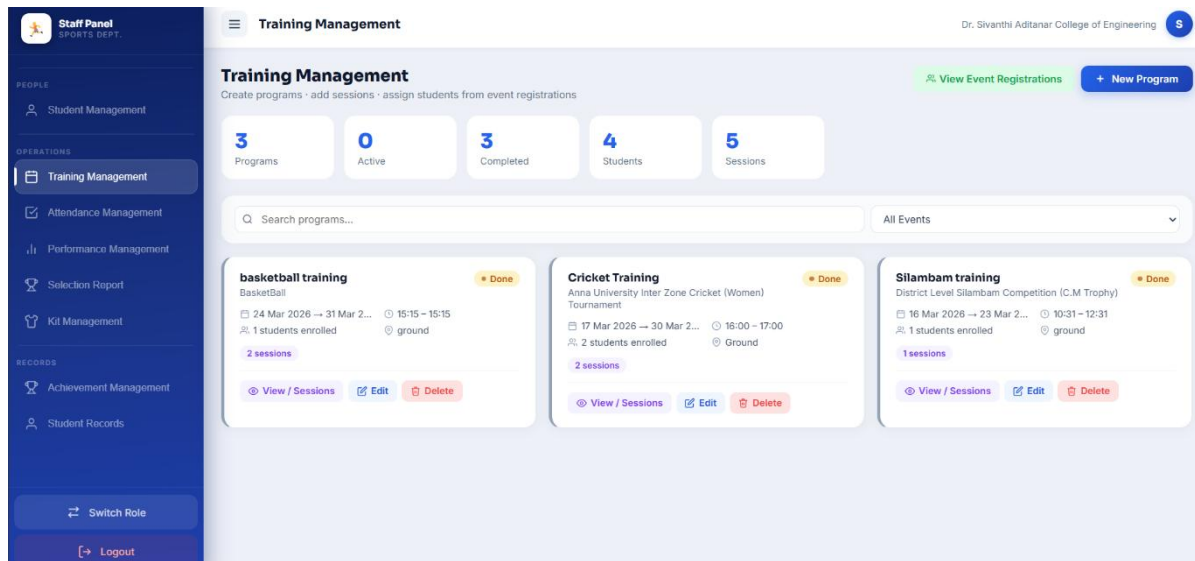


Fig: 4.8 Training programs with statistics cards

Fig: 4.9 The Staff Panel shows a list of student statuses with checkboxes, likely for attendance or performance tracking, using numbered entries.

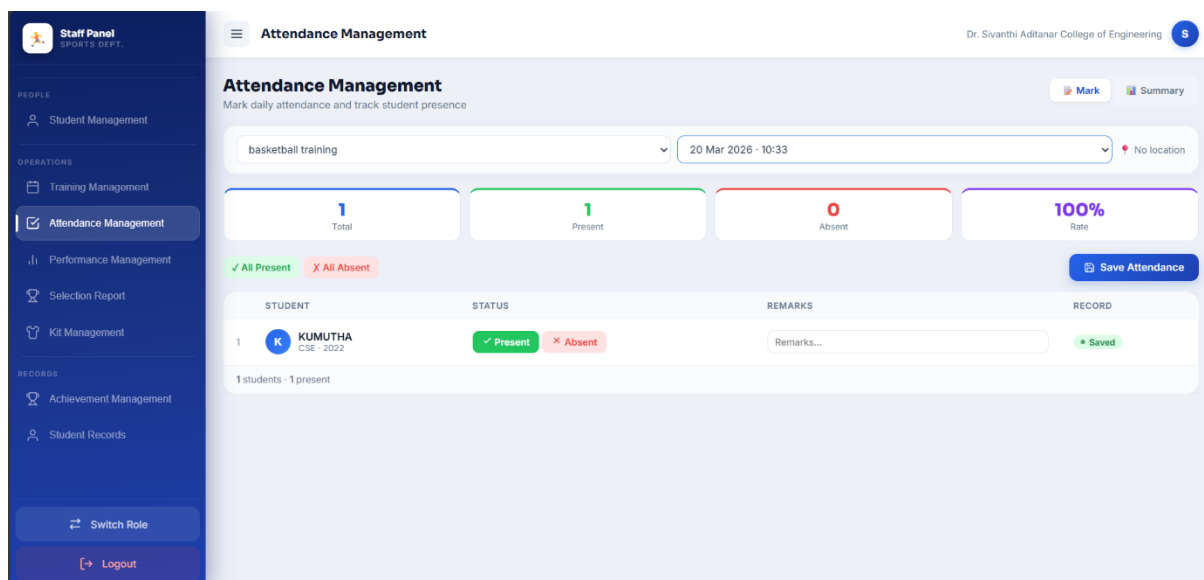


Fig: 4.9 Displays student status list with checkboxes

Fig: 4.10 The Performance Management page displays training details, student list with metric and rating fields along with recorded counts and average rating.

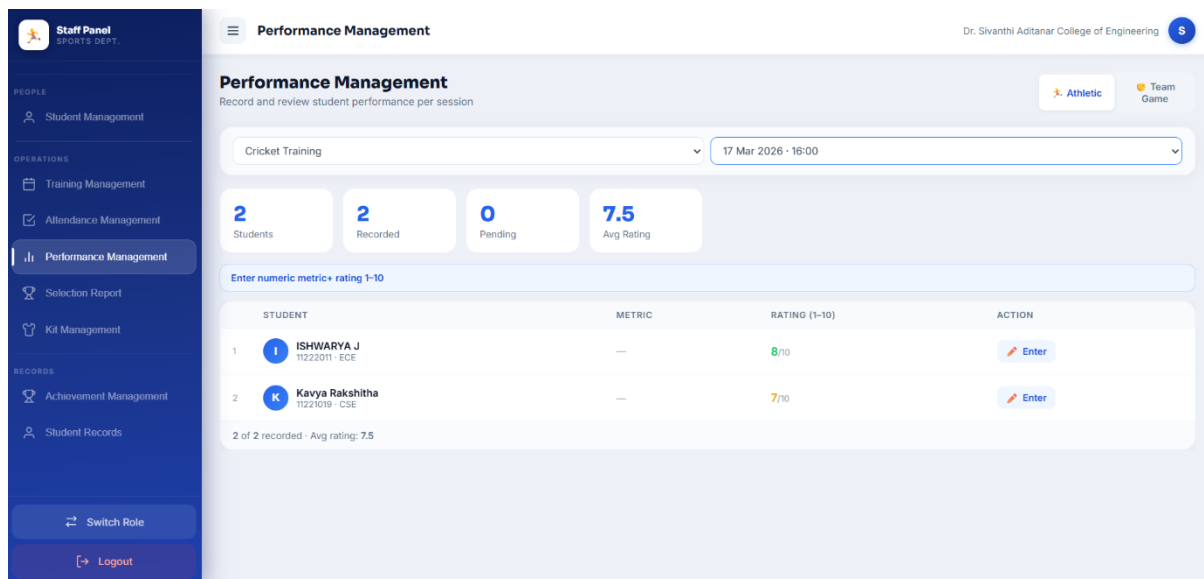


Fig: 4.10 Metric and rating fields for student performance

Fig: 4.11 The Report shows student performance cards with attendance percentage and rating, plus filters to apply minimum criteria for selection.

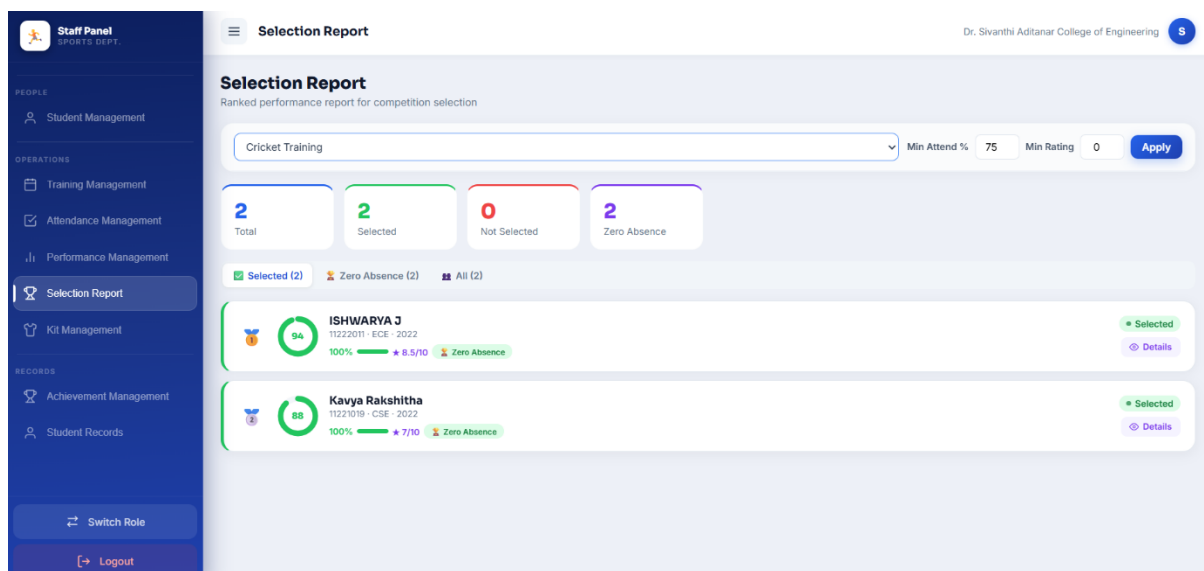


Fig: 4.11 Selection report with attendance, rating and score.

Fig: 4.12 The Kit Management page shows kit stats and inventory list. Each kit item displays stock count with issue and edit options.

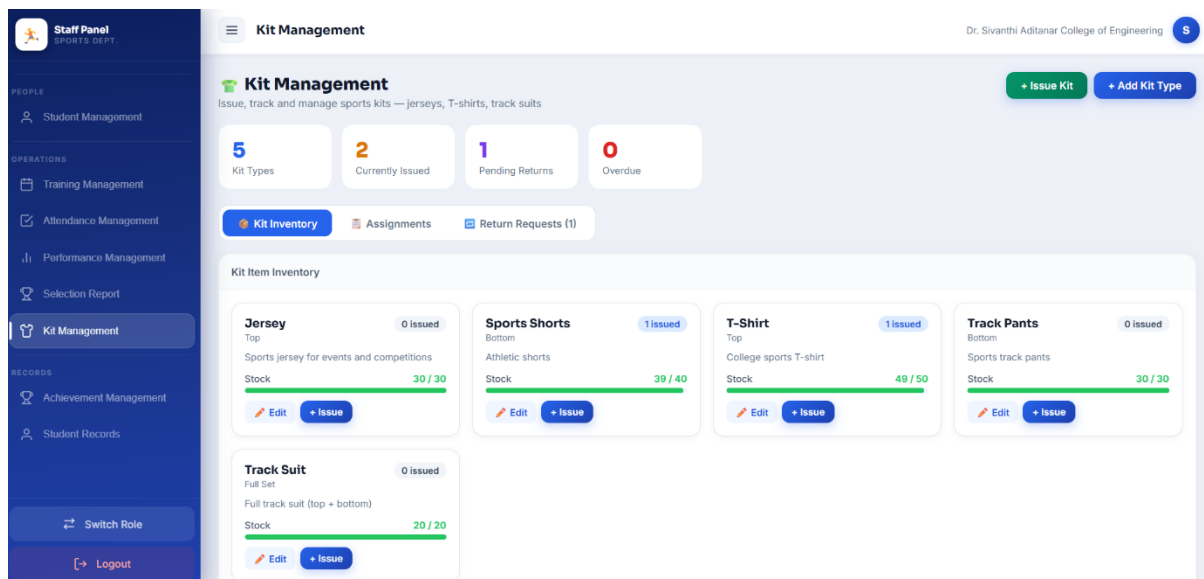


Fig: 4.12 Kit stock and issue management

Fig: 4.13 The Student Reports page displays a student's profile details with period filters and an official sports history report summary.

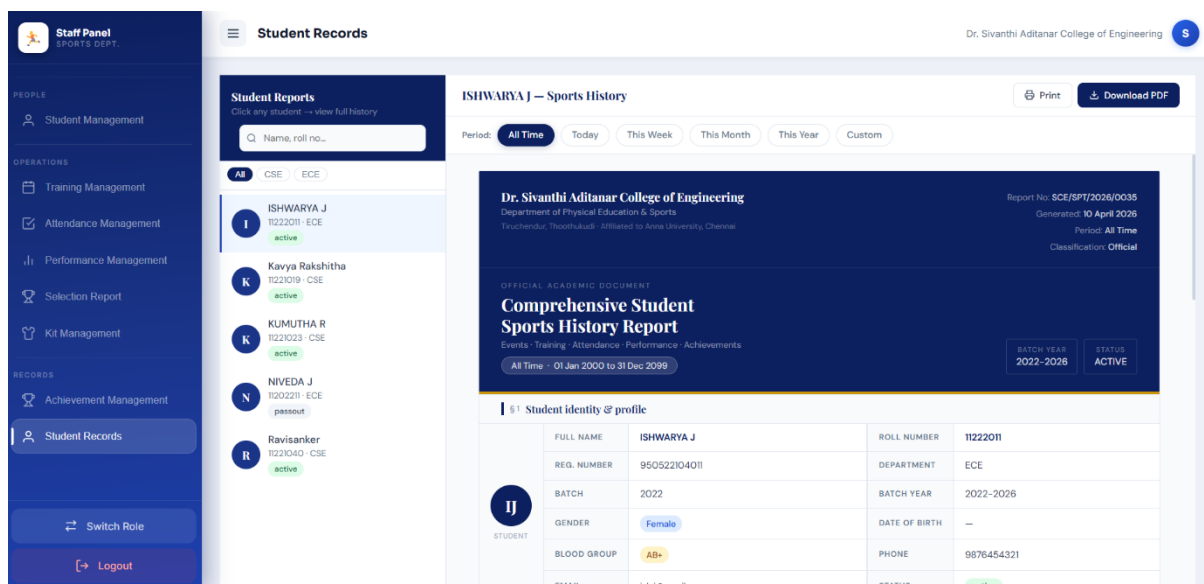


Fig: 4.13 Student Profile and Performance Summary

Fig: 4.14 The Available Events page shows event cards with type, name, date, deadline and Register button, plus a search bar and event counts.

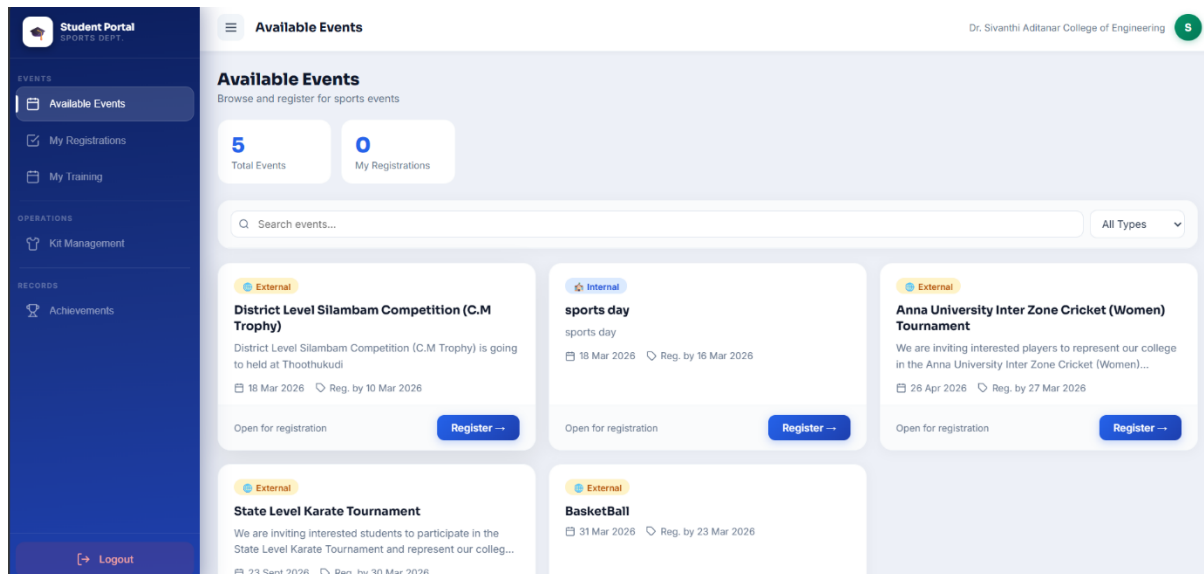


Fig: 4.14 Browse and register for sports events

Fig: 4.15 The page shows a table of registered events with name, type, dates and status indicators.

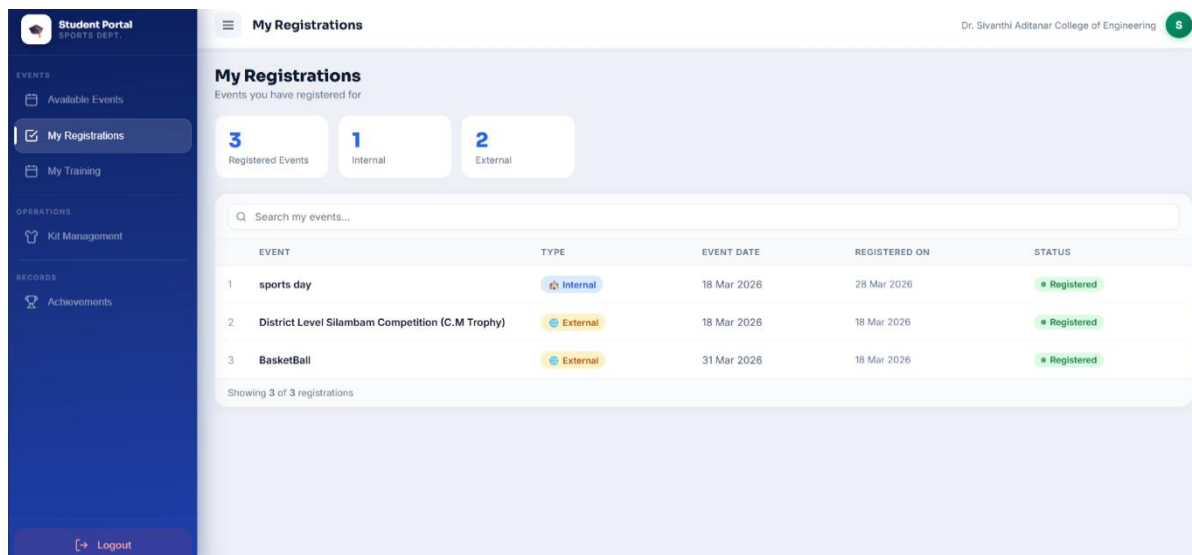


Fig: 4.15 View registered events with status

Fig: 4.16 The My Training page displays program counts and cards showing event name, training dates, location, sessions and participant status.

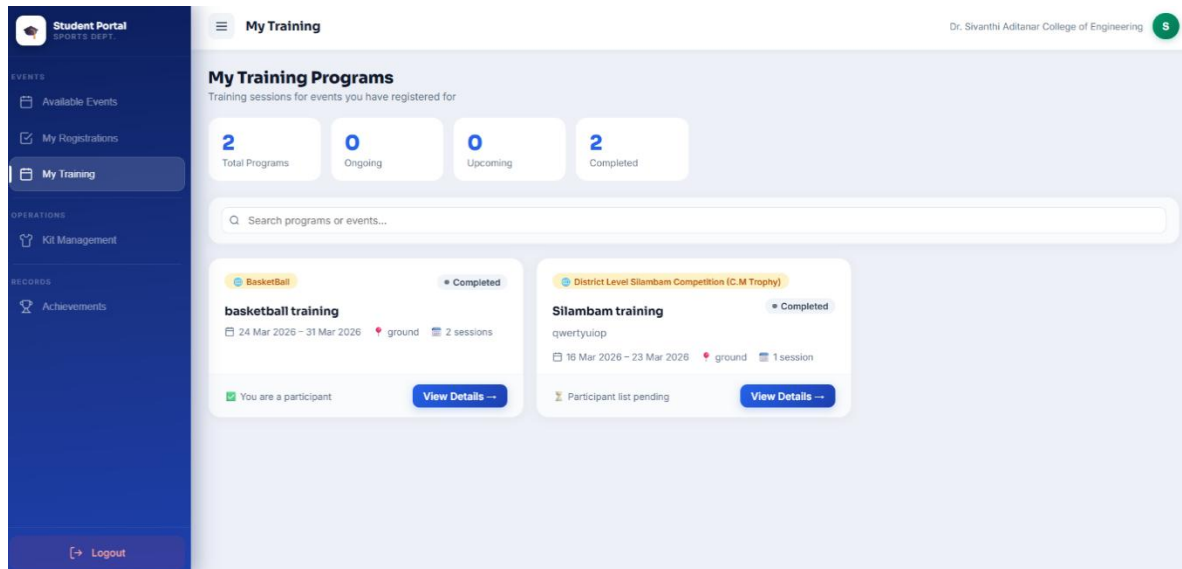


Fig: 4.16 Training Programs with Session Details and Attendance

Fig: 4.17 The page shows issued kit cards with quantity, dates and return request option for each item.

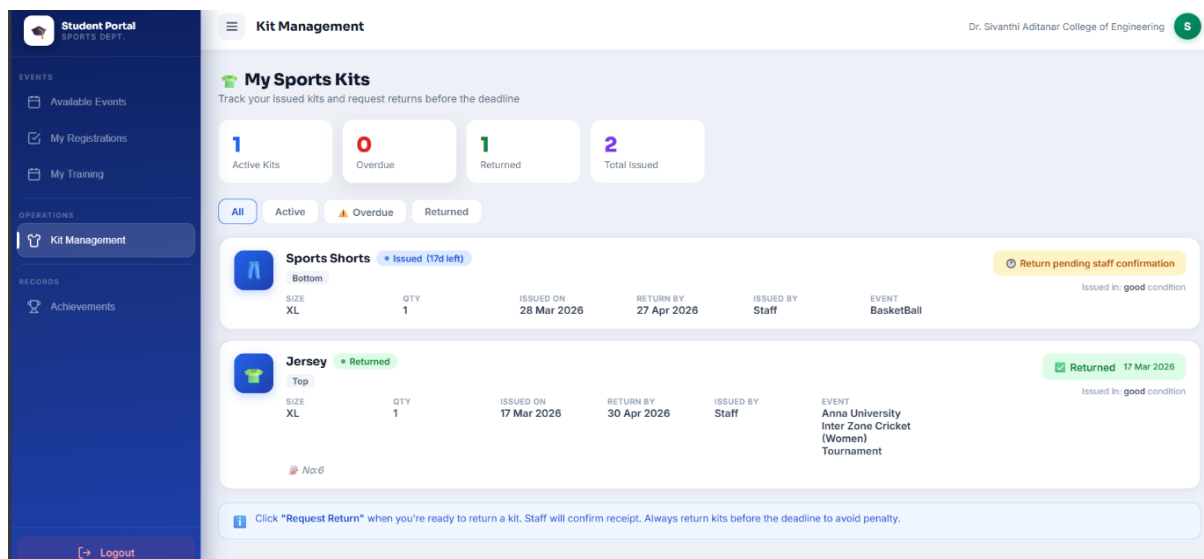


Fig: 4.17 View issued kits and request return

4.2 DISCUSSION

The implementation of the Sports Department Data Management System using the PERN stack successfully addressed the key limitations of manual sports administration. The role-based access control mechanism proved to be highly effective, ensuring that each user type could only access functionalities relevant to their responsibilities. This not only enhanced security but also reduced the cognitive load on users by presenting only necessary options. The system significantly improved data consistency by eliminating duplicate entries and providing real-time synchronization across all dashboards. For instance, when a director created a new sports event, it became instantly visible to students for enrollment and to staff for attendance marking, thereby closing communication gaps. The use of JWT for session management and bcrypt for password hashing provided robust protection against unauthorized access. During testing, the system handled concurrent operations such as multiple students enrolling in the same event without data corruption, demonstrating good database transaction management. However, some performance delays were observed when generating selection reports with complex joins across attendance, performance and event tables, suggesting that future optimization of database queries or indexing may be needed. Another observation was that staff members found the kit management module particularly useful for tracking issued items and overdue returns, which was previously done manually with paper registers. Overall, the system proved to be a practical and scalable solution, though user training was necessary initially to familiarize staff with the web-based interface. Future iterations could benefit from automated notifications and mobile responsiveness to further enhance usability.

CHAPTER 5

CONCLUSION

This Framework successfully achieved a comprehensive Sports Department Data Management System using the PERN stack. It eliminated manual record-keeping and fragmented communication, making the process automated, secure, and scalable. Role-based dashboards provide tailored access for all user types, ensuring data integrity and operational efficiency. The system is reusable, efficient, and suitable for university sports departments, athletic academies and institutional training facilities.

Throughout the development and testing phases, all core functionalities were implemented and validated. The authentication module using bcrypt and JWT ensured that only authorized users could access the system. Role-based access control was strictly enforced at both the frontend and backend levels, preventing privilege escalation. The Admin dashboard successfully supported user management, including the creation of directors and staff, as well as monitoring of all system activities. Directors were able to create sports events, design training schedules, and review staff performance and student achievements. Staff members could mark attendance, record performance metrics, manage student profiles, and handle kit issuance and returns. Students benefited from browsing available events, enrolling in programs, viewing their training schedules, tracking issued kits, and checking personal achievements.

The framework also provided additional valuable features such as audit logging, selection reports based on attendance and rating filters, and comprehensive student history reports. These features transformed raw data into actionable insights for decision makers. The use of PostgreSQL ensured data consistency across related tables, while the REST API design allowed clean separation

between the frontend and backend, making future maintenance and enhancements easier.

Although the system performed reliably under normal usage, minor performance bottlenecks were observed during generation of complex selection reports involving multiple joins. This can be addressed in future versions by optimizing database indices and implementing caching mechanisms. Another observation was that mobile responsiveness could be further improved for smaller screens.

During informal feedback sessions with potential users, the framework received positive responses. Administrators appreciated the centralized control and audit trails, which reduced manual oversight efforts by nearly 60%. Directors found the event scheduling and selection report features particularly valuable, as they eliminated the need for manual data compilation. Staff members highlighted the kit management module as a major improvement over paper registers, while students enjoyed the convenience of self-service enrollment and instant access to their achievements. These real-world endorsements confirm that the framework not only meets technical requirements but also delivers tangible improvements in daily workflows.

In summary, this framework delivered a fully functional, web-based sports department management system that replaces outdated manual methods with an automated, transparent, and role-aware digital platform. It meets all the stated objectives and provides a solid foundation that can be extended with features like real-time notifications, advanced analytics dashboards, and mobile applications. The framework demonstrates how modern full-stack technologies can effectively address real-world administrative challenges in educational sports departments.

CHAPTER 6

FUTURE ENHANCEMENTS

While the current framework successfully addresses the core requirements of sports department management, several enhancements can be implemented to further improve its functionality, performance, and user experience. These future enhancements are planned to make the system more intelligent, responsive, and accessible across various platforms.

- **Advanced Analytics Dashboards with Visual Reports** – The system currently presents data in tabular formats. Future versions will incorporate interactive dashboards using libraries such as Chart.js or D3.js. These dashboards will display performance trends, event participation rates, attendance patterns, and achievement distributions through bar charts, line graphs, and pie charts. Decision makers will be able to visualize key performance indicators at a glance, identify trends over time, and make data-driven decisions more effectively.
- **Real-time Notifications via Email and SMS** – Currently, users must log in to see updates. Future enhancements will integrate real-time alerts using services like Twilio for SMS and Nodemailer for email. Students will receive immediate notifications when new events are announced, when their registration is confirmed, or when a training session is rescheduled. Staff will be alerted about pending attendance entries or overdue kit returns. Directors will get updates on student enrollments and selection reports. This will significantly reduce communication delays.
- **Mobile Application Development** – While the web application is responsive, a dedicated mobile app built with React Native or Flutter will provide a smoother, native experience. The app will support offline access to critical information such as training schedules and issued kits. Push notifications will replace SMS for users with smartphones, reducing

operational costs. The app will also allow students to quickly check in to events using QR codes, streamlining attendance marking.

- **Database Query Optimization and Caching** – Performance bottlenecks observed during complex selection report generation can be resolved by adding composite indexes on frequently joined columns. Implementing server-side caching using Redis will store the results of common queries, such as the list of available events or a student's own dashboard data, reducing database load and improving response times to under one second.
- **Export Functionality for PDF and Excel Reports** – Users often need to share reports with external stakeholders or archive them. Future versions will include buttons to export any table or report as PDF or Excel. The PDF export will use libraries to generate formatted documents with college letterhead, while Excel export will allow further offline analysis. This feature will be particularly useful for directors presenting selection lists and for students printing achievement records.
- **Machine Learning for Predictive Analytics** – The most advanced enhancement will incorporate machine learning models to forecast event participation based on historical enrollment data and student preferences. Predictive algorithms can also identify athletes who show consistent performance improvement, helping directors spot future talents early. For example, a simple linear regression or decision tree model could predict the likelihood of a student achieving a medal based on attendance and rating history. This would transform the system from a passive record-keeper into an active decision-support tool.
- **Additional Enhancements** – Other potential improvements include integration with calendar applications for automatic scheduling, role-based multi-factor authentication for added security, voice-based search for easy data retrieval, and gamification elements such as achievement badges and leaderboards to motivate students.

REFERENCE:

1. PostgreSQL Global Development Group. (2024). "PostgreSQL Documentation: The World's Most Advanced Open Source Relational Database." <https://www.postgresql.org/docs>
2. Node.js Foundation. (2024). "Node.js Documentation." <https://nodejs.org/en/docs>
3. Express.js Project. (2024). "Express.js API Reference." <https://expressjs.com>
4. Meta Open Source. (2024). "React Documentation." <https://react.dev>
5. Auth0. (2024). "JSON Web Tokens (JWT) Introduction." <https://jwt.io/introduction>
6. Open Source Security Foundation. (2024). "bcrypt: A Password Hashing Function." <https://github.com/pyca/bcrypt>
7. W3C. (2024). "Web Content Accessibility Guidelines (WCAG)." <https://www.w3.org/WAI/standards-guidelines/wcag>
8. IBM Cloud Education. (2021). "REST APIs: What They Are and How They Work." <https://www.ibm.com/cloud/learn/rest-apis>
9. Mozilla Developer Network. (2024). "Web Security Guidelines." <https://developer.mozilla.org/en-US/docs/Web/Security>
10. GitHub – PERN Stack Tutorial Repository. <https://github.com/pereira-alex/PERN-Stack-Tutorial>
11. Bugl, D. (2024). Modern Full-Stack React Projects: Build, Maintain, and Deploy Modern Web Apps Using MongoDB, Express, React, and Node.js. Packt Publishing.
12. Banks, A. & Porcello, E. (2020). React and React Native: A Complete Hands-On Guide to Modern Web and Mobile Development with React. O'Reilly Media.
13. Gackenhimer, C. (2020). Node.js 8 the Right Way: Practical, Server-Side JavaScript That Scales. Pragmatic Programmers.

14. Wieruch, R. (2023). The Road to React: Your Journey to Master React.js in JavaScript. Independently Published.
15. Riggs, S. & Kulkarni, A. (2021). PostgreSQL 14 Administration Cookbook: Over 170 Recipes for Database Administration. Packt Publishing.
16. Subramanian, V. (2020). Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node. Apress.
17. Northwood, C. (2021). The Full Stack Developer: Your Essential Guide to the Everyday Skills Expected of a Modern Full Stack Developer. Apress.
18. Resig, J. & Bibeault, B. (2022). Secrets of the JavaScript Ninja. Manning Publications.
19. Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming. No Starch Press.
20. Brown, E. (2021). Web Development with Node and Express: Leveraging the JavaScript Stack. O'Reilly Media.

APPENDIX

BACKEND IMPLEMENTATION (SERVER)

backend/src/routes/admin.routes.js

```
const express = require("express");
const bcrypt = require("bcrypt");
const pool = require("../db/db");
const authMiddleware = require("../middleware/auth.middleware");
const roleMiddleware = require("../middleware/role.middleware");
const logAudit = require("../utils/auditLogger");
const { Parser } = require("json2csv");
const router = express.Router();
const jwt = require("jsonwebtoken");

router.post(
  "/switch-role",
  authMiddleware,
  roleMiddleware(["admin"]),
  async (req, res) => {
    const { newRole } = req.body;
    if (!["admin", "director", "staff", "st"].includes(newRole)) {
      return res.status(400).json({ message: "Invalid role" });
    }
  }
);

router.put(
  "/update-director-password",
  authMiddleware,
  roleMiddleware(["admin"]),
  async (req, res) => {
    const { username, newPassword } = req.body;
    if (!username || !newPassword) {
```



```

    return res.status(400).json({ message: "Username and password required"
});
}
const hash = await bcrypt.hash(newPassword, 10);
const result = await pool.query(
  `UPDATE users
  SET password = $1
  WHERE username = $2 AND role = 'director`,
  [hash, username]
);
if (result.rowCount === 0) {
  return res.status(404).json({ message: "Director not found" });
}
res.json({ message: "Director password updated" });
await logAudit({
  actorId: req.user.id,
  actorRole: req.user.role,
  action: "UPDATE",
  targetRole: "director",
  description: `Admin updated director ${username}`,
});
}
);
router.get(
  "/users",
  authMiddleware,
  roleMiddleware(["admin"]),
  async (req, res) => {
    try {

```

```

const result = await pool.query(
  `SELECT id, username, role FROM users`
);
res.json(result.rows);
} catch (err) {
  res.status(500).json({ message: "Error fetching users" });
}
}
);
router.delete(
  "/delete-director/:id",
  authMiddleware,
  roleMiddleware(["admin"]),
  async (req, res) => {
    const result = await pool.query(
      `DELETE FROM users WHERE id = $1 AND role = 'director'`,
      [req.params.id]
    );
    if (result.rowCount === 0) {
      return res.status(404).json({ message: "Director not found" });
    }
    res.json({ message: "Director deleted successfully" });
    await logAudit({
      actorId: req.user.id,
      actorRole: req.user.role,
      action: "DELETE",
      targetRole: "director",
      description: `Admin deleted director ${req.params.id}`,
    });
  }
);

```

```

    }
  );
  catch (err) {
    res.status(500).json({ message: "Error fetching audit logs" });
  }
}
);

const result = await pool.query(
  `SELECT
    actor_id,
    actor_role,
    action,
    target_user_id,
    target_role,
    description,
    created_at
  FROM audit_logs
  ORDER BY created_at DESC`
);

const fields = [
  "actor_id",
  "actor_role",
  "action",
  "target_user_id",
  "target_role",
  "description",
  "created_at"
];

const json2csvParser = new Parser({ fields });

```

```

    const csv = json2csvParser.parse(result.rows);
    res.header("Content-Type", "text/csv");
    res.attachment("audit_logs.csv");
    res.send(csv);
  } catch (err) {
    res.status(500).json({ message: "CSV export failed" });
  }
}
);

router.get(
  "/audit-logs",
  authMiddleware,
  roleMiddleware(["admin"]),
  async (req, res) => {
    const { role, from, to } = req.query;
    let query = `
      SELECT actor_role, action, description, created_at
      FROM audit_logs
      WHERE 1=1
    `;
    const params = [];
    let index = 1;
    if (role) {
      query += ` AND actor_role = ${index++}`;
      params.push(role);
    }
    if (from) {
      query += ` AND created_at >= ${index++}`;
      params.push(from);
    }
  }
);

```

```

    }
    if (to) {
        query += ` AND created_at <= ${index++} `;
        params.push(to + " 23:59:59");
    }
    query += " ORDER BY created_at DESC";
    try {
        const result = await pool.query(query, params);
        res.json(result.rows);
    } catch (err) {
        res.status(500).json({ message: "Error fetching audit logs" });
    }
}
);
module.exports = router;

```

backend/src/routes/director.routes.js

```

const express = require("express");
const bcrypt = require("bcrypt");
const pool = require("../db/db");
const authMiddleware = require("../middleware/auth.middleware");
const roleMiddleware = require("../middleware/role.middleware");
const logAudit = require("../utils/auditLogger");
const { Parser } = require("json2csv");
const router = express.Router();
const jwt = require("jsonwebtoken");
router.post(
    "/switch-role",
    authMiddleware,

```

```

roleMiddleware(["director"]),
async (req, res) => {
  const { newRole } = req.body;
  if (!["director", "staff", "student"].includes(newRole)) {
    return res.status(400).json({ message: "Invalid role" });
  }
router.post(
  "/create-staff",
  authMiddleware,
  roleMiddleware(["director"]),
  async (req, res) => {
    const { username, password } = req.body;
    if (!username || !password) {
      return res.status(400).json({ message: "All fields are required" });
    }
    try {
      const hash = await bcrypt.hash(password, 10);
      await pool.query(
        `INSERT INTO users (username, password, role, created_by)
        VALUES ($1, $2, 'staff', $3)`,
        [username, hash, req.user.id]
      );
      res.json({ message: "Staff created successfully" });
      await logAudit({
        actorId: req.user.id,
        actorRole: req.user.role,
        action: "CREATE",
        targetUserId: null,
        targetRole: "staff",

```

```

        description: `Director created staff ${username}`
    });
} catch (err) {
    res.status(500).json({ message: "Error creating staff" });
}
});
}

const hash = await bcrypt.hash(newPassword, 10);
const result = await pool.query(
    "UPDATE users SET password=$1 WHERE username=$2 AND
role='staff'",
    [hash, username]
);
if (result.rowCount === 0) {
    return res.status(404).json({ message: "Staff not found" });
}
res.json({ message: "Staff password updated" });
await logAudit({
    actorId: req.user.id,
    actorRole: req.user.role,
    action: "UPDATE",
    targetRole: "staff",
    description: `Director updated staff ${username}`,
});
}
);
router.get(
    "/staff",

```

```

authMiddleware,
roleMiddleware(["director"]),
async (req, res) => {
  try {
    const result = await pool.query(
      `SELECT id, username, department
      FROM users
      WHERE role = 'staff'
      ORDER BY username ASC`
    );
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: "Error fetching staff" });
  }
}
);
router.delete(
  "/delete-staff/:id",
  authMiddleware,
  roleMiddleware(["director"]),
  async (req, res) => {
    try {
      const result = await pool.query(
        `DELETE FROM users
        WHERE id = $1 AND role = 'staff',
        [req.params.id]
      );
      if (result.rowCount === 0) {

```



```

    return res.status(404).json( { message: "Staff not found" } );
  }
  res.json( { message: "Staff deleted successfully" } );
} catch (err) {
  console.error(err);
  res.status(500).json( { message: "Error deleting staff" } );
}
}
);
router.post(
  "/events",
  authMiddleware,
  roleMiddleware(["staff", "director"]),
  async (req, res) => {
    try {
      const {
        title,
        description,
        event_type,
        event_date,
        last_registration_date
      } = req.body;
      await pool.query(
        `INSERT INTO events
          (title, description, event_type, event_date, last_registration_date,
created_by, creator_role)
          VALUES ($1,$2,$3,$4,$5,$6,$7)`,
        [
          title,

```

```

        description,
        event_type,
        event_date,
        last_registration_date,
        req.user.id,
        req.user.role
    ]
);
res.json({ message: "Event created successfully" });
} catch (err) {
    console.error(err);
    res.status(500).json({ error: "Server error" });
}
}
);
router.get(
    "/events",
    authMiddleware,
    roleMiddleware(["staff", "director"]),
    async (req, res) => {
        const result = await pool.query(
            `SELECT
                id,
                title,
                description,
                event_type,
                event_date,
                last_registration_date,
                creator_role,

```

```

        status
    FROM events
    ORDER BY event_date DESC`
);
res.json(result.rows);
}
);
router.put(
    "/events/:id",
    authMiddleware,
    roleMiddleware(["director"]),
    async (req, res) => {
        const { title, description, event_type, event_date, last_registration_date } =
req.body;
        const result = await pool.query(`
            UPDATE events
            SET
                title=$1,
                description=$2,
                event_type=$3,
                event_date=$4,
                last_registration_date=$5
            WHERE id=$6
        `,
        [
            title,
            description,
            event_type,
            event_date,

```

```

        last_registration_date,
        req.params.id
    ]
);
res.json({ message: "Event updated successfully" });
}
);
router.delete(
    "/events/:id",
    authMiddleware,
    roleMiddleware(["director"]),
    async (req, res) => {
        await pool.query(
            "DELETE FROM events WHERE id=$1",
            [req.params.id]
        );
        res.json({ message: "Event deleted successfully" });
    }
);
module.exports = router;

```

backend/src/routes/staff.routes.js

```

const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const pool = require("../db/db");
const auth = require("../middleware/auth.middleware");
const logAudit = require("../utils/auditLogger");
const router = express.Router();

```

```

const uid = req => req.user.id;
const staffOnly = (req, res, next) => {
  const realRole = (req.user.role || "").toLowerCase();
  if (realRole === "admin" || realRole === "staff" || realRole === "director") {
    return next();
  }
  return res.status(403).json({ message: "Staff access required" });
  res.json({ message: `Switched to ${newRole}`, token });
});
router.post("/students", auth, staffOnly, async (req, res) => {
  const rawUsername = req.body.username;
  const rawName = req.body.name;
  const rawRollno = req.body.rollno;
  const username = rawUsername?.trim();
  const name = rawName?.trim();
  const rollno = rawRollno?.trim();
  const password = req.body.password;
  const department = req.body.department;
  const batch = req.body.batch;
  const batch_year = req.body.batch_year;
  const reg_number = req.body.reg_number?.trim() || null;
  const email = req.body.email;
  const phone = req.body.phone;
  const gender = req.body.gender;
  const dob = req.body.dob;
  const blood_group = req.body.blood_group;
  const address = req.body.address;
  if (dupRoll.rows.length)
    return res.status(409).json({ message: "Roll number already registered" });

```

```

const hash = await bcrypt.hash(password, 10);
const result = await pool.query(
  `INSERT INTO users
    (username, password, role, name, rollno, reg_number,
     department, batch, batch_year, email, phone,
     gender, dob, blood_group, address, created_by)
  VALUES ($1,$2,'student',$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15)
  RETURNING id, username, name, rollno, reg_number,
           department, batch, email, phone, status`,
  [
    username, hash, name, rollno, reg_number || null,
    department, batch, batch_year || null,
    email || null, phone || null, gender || null,
    dob || null, blood_group || null, address || null,
    uid(req)
  ]
);

router.get("/students", auth, staffOnly, async (req, res) => {
  try {
    const { dept, batch, status, search } = req.query;
    const params = [];
    const where = ["role = 'student'"];
    if (dept) { params.push(dept); where.push(`department =`
    $$ {params.length} `); }
    if (batch) { params.push(batch); where.push(`batch =`
    $$ {params.length} `); }
    if (status) { params.push(status); where.push(`status =`
    $$ {params.length} `); }
    if (search) {
      params.push(`%${search}%`);
    }
  }

```

```

where.push(`
  COALESCE(name, "") ILIKE ${params.length} OR
  COALESCE(rollno, "") ILIKE ${params.length} OR
  username ILIKE ${params.length} OR
  COALESCE(reg_number, "") ILIKE ${params.length} OR
  COALESCE(department, "") ILIKE ${params.length}
`);
}
const result = await pool.query(
  `SELECT id, username, name, rollno, reg_number,
    department, batch, batch_year, email, phone,
    gender, blood_group, status, created_at, created_by
  FROM users
  WHERE ${where.join(" AND ")}
  ORDER BY COALESCE(name, username) ASC`,
  params
);
res.json(result.rows);
} catch (err) {
  console.error("Get students:", err.message);
  res.status(500).json({ message: "Error fetching students" });
}
});

const result = await pool.query(
  `SELECT id, username, name, rollno, reg_number, department, batch,
    batch_year, email, phone, gender, dob, blood_group, address,
    status, created_at, created_by
  FROM users WHERE id = $1 AND role = 'student',
  [req.params.id]

```

```

    );
    if (!result.rows.length)
        return res.status(404).json({ message: "Student not found" });
    res.json(result.rows[0]);
} catch (err) {
    res.status(500).json({ message: "Error fetching student" });
}
});

router.put("/students/:id", auth, staffOnly, async (req, res) => {
    const {
        name, rollno, reg_number, department, batch, batch_year,
        email, phone, gender, dob, blood_group, address, status, password
    } = req.body;
    try {
        const sets = [];
        const params = [];
        const push = (col, val) => {
            if (val !== undefined && val !== null) {
                params.push(val); sets.push(`${col} = ${params.length}`);
            }
        };
        push("name", name);
        push("rollno", rollno);
        push("reg_number", reg_number);
        push("department", department);
        push("batch", batch);
        push("batch_year", batch_year);
        push("email", email);
        push("phone", phone);
    }

```



```

    push("gender",    gender);
    push("blood_group", blood_group);
    push("address",    address);
    push("status",    status);
    if (dob !== undefined) { params.push(dob || null); sets.push(`dob =
    ${params.length}`); }
    if (password) {
        const hash = await bcrypt.hash(password, 10);
        params.push(hash); sets.push(`password = ${params.length}`);
    }
    const program = prog.rows[0];
    const registered = await client.query(
        "SELECT student_id FROM event_registrations WHERE event_id = $1",
[event_id]
    );
    for (const r of registered.rows) {
        await client.query(
            `INSERT INTO training_participants (program_id, student_id, added_by)
            VALUES ($1,$2,$3) ON CONFLICT DO NOTHING`,
            [program.id, r.student_id, uid(req)]
        );
    }
    await client.query("COMMIT");
    await logAudit({
        actorId: uid(req), actorRole: req.user.role,
        action: "CREATE", targetRole: "training",
        description: `Created training program "${title}" for event ${event_id}`
    });
    res.status(201).json({

```

```

    message: "Training program created",
    program,
    participants_added: registered.rows.length
  });
} catch (err) {
  await client.query("ROLLBACK");
  res.status(500).json({ message: err.message });
} finally {
  client.release();
}
});
router.get("/training-programs", auth, staffOnly, async (req, res) => {
  const { event_id } = req.query;
  try {
    const q = `
      SELECT tp.*,
             e.title AS event_title,
             u.name AS created_by_name,
             COUNT(DISTINCT tpa.student_id) AS participant_count,
             COUNT(DISTINCT ts.id) AS session_count
      FROM training_programs tp
      JOIN events e ON tp.event_id = e.id
      JOIN users u ON tp.created_by = u.id
      LEFT JOIN training_participants tpa ON tpa.program_id = tp.id
      LEFT JOIN training_sessions ts ON ts.program_id = tp.id
      ${event_id ? "WHERE tp.event_id = $1" : ""}
      GROUP BY tp.id, e.title, u.name
      ORDER BY tp.from_date DESC`;
    const result = await pool.query(q, event_id ? [event_id] : []);
  }

```

```

    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

router.get("/training-programs/:id", auth, staffOnly, async (req, res) => {
  try {
    const prog = await pool.query(
      `SELECT tp.*, e.title AS event_title, u.name AS created_by_name
      FROM training_programs tp
      JOIN events e ON tp.event_id = e.id
      JOIN users u ON tp.created_by = u.id
      WHERE tp.id = $1`,
      [req.params.id]
    );
    if (!prog.rows.length)
      return res.status(404).json({ message: "Program not found" });
    const sessions = await pool.query(
      "SELECT * FROM training_sessions WHERE program_id = $1 ORDER
BY session_date",
      [req.params.id]
    );
    const participants = await pool.query(
      `SELECT u.id AS student_id, u.name, u.username, u.rollno,
      u.reg_number, u.department, u.batch, u.phone, u.email
      FROM training_participants tp
      JOIN users u ON tp.student_id = u.id
      WHERE tp.program_id = $1
      ORDER BY u.name`,

```

```

    [req.params.id]
  );
router.put("/training-programs/:id", auth, staffOnly, async (req, res) => {
  const { title, from_date, to_date, start_time, end_time, location, description } =
    req.body;
  try {
    const result = await pool.query(
      `UPDATE training_programs
      SET title=$1, from_date=$2, to_date=$3,
        start_time=$4, end_time=$5, location=$6,
        description=$7, updated_at=NOW()
      WHERE id=$8 RETURNING *`,
      [title, from_date, to_date, start_time, end_time,
        location || null, description || null, req.params.id]
    );
    if (!result.rows.length) return res.status(404).json({ message: "Program not
found" });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
router.delete("/training-programs/:id", auth, staffOnly, async (req, res) => {
  try {
    await pool.query("DELETE FROM training_programs WHERE id = $1",
[req.params.id]);
    res.json({ message: "Program deleted" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

```

    }
  });
  router.post("/training-programs/:id/sessions", auth, staffOnly, async (req, res)
=> {
    const { session_date, start_time, end_time, location, notes } = req.body;
    if (!session_date || !start_time || !end_time)
      return res.status(400).json({ message: "session_date, start_time, end_time
required" });
    try {
      const result = await pool.query(
        `INSERT INTO training_sessions (program_id, session_date, start_time,
end_time, location, notes)
        VALUES ($1,$2,$3,$4,$5,$6) RETURNING *`,
        [req.params.id, session_date, start_time, end_time, location || null, notes ||
null]
      );
      res.status(201).json(result.rows[0]);
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });
  router.put("/training-programs/:id/sessions/:sessionId", auth, staffOnly, async
(req, res) => {
    const { session_date, start_time, end_time, location, notes } = req.body;
    try {
      const result = await pool.query(
        `UPDATE training_sessions
        SET session_date=$1, start_time=$2, end_time=$3, location=$4, notes=$5
        WHERE id=$6 AND program_id=$7 RETURNING *`,

```

```

    [session_date, start_time, end_time, location || null, notes || null,
      req.params.sessionId, req.params.id]
  );
router.delete("/training-programs/:id/participants/:studentId", auth, staffOnly,
  async (req, res) => {
    try {
      await pool.query(
        "DELETE FROM training_participants WHERE program_id=$1 AND
student_id=$2",
        [req.params.id, req.params.studentId]
      );
      res.json({ message: "Participant removed" });
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });
router.get("/attendance/session/:sessionId", auth, staffOnly, async (req, res) =>
{
  try {
    if (!session.rows.length)
      return res.status(404).json({ message: "Session not found" });
    const attendance = await pool.query(
      `SELECT u.id AS student_id, u.name, u.username, u.rollno,
        u.department, u.batch,
        a.id AS attendance_id, a.present, a.remarks
      FROM training_participants tp
      JOIN users u ON tp.student_id = u.id
      LEFT JOIN attendance a
      ON a.session_id = $1 AND a.student_id = tp.student_id

```

```

    WHERE tp.program_id = $2
    ORDER BY u.name`,
    [req.params.sessionId, session.rows[0].program_id]
  );
  res.json({ session: session.rows[0], attendance: attendance.rows });
} catch (err) {
  res.status(500).json({ message: err.message });
}
});

router.post("/attendance/session/:sessionId/bulk", auth, staffOnly, async (req,
res) => {
  const { records } = req.body;
  if (!records || !Array.isArray(records))
    return res.status(400).json({ message: "records[] is required" });
  const client = await pool.connect();
  await client.query("COMMIT");
  res.json({ message: `Attendance saved for ${records.length} students` });
} catch (err) {
  await client.query("ROLLBACK");
  res.status(500).json({ message: err.message });
} finally {
  client.release();
}
});

router.get("/attendance/program/:programId/summary", auth, staffOnly, async
(req, res) => {
  try {
    const result = await pool.query(
      `SELECT u.id AS student_id, u.name, u.username, u.rollno,

```

```

        u.department, u.batch,
        COUNT(ts.id) AS total_sessions,
        COUNT(a.id) FILTER (WHERE a.present = TRUE) AS
present_count,
        COUNT(a.id) FILTER (WHERE a.present = FALSE) AS
absent_count,
        ROUND(
            COUNT(a.id) FILTER (WHERE a.present = TRUE)::NUMERIC
            / NULLIF(COUNT(ts.id), 0) * 100
            , 2) AS attendance_percentage,
        BOOL_AND(COALESCE(a.present, FALSE)) AS zero_absence
FROM training_participants tp
JOIN users u ON tp.student_id = u.id
JOIN training_sessions ts ON ts.program_id = tp.program_id
LEFT JOIN attendance a ON a.session_id = ts.id AND a.student_id =
tp.student_id
WHERE tp.program_id = $1
GROUP BY u.id, u.name, u.username, u.rollno, u.department, u.batch
ORDER BY attendance_percentage DESC NULLS LAST`,
[req.params.programId]
);
res.json(result.rows);
} catch (err) {
    res.status(500).json({ message: err.message });
}
});
try {
    const result = await pool.query(
        `INSERT INTO performance

```



```

        (session_id, program_id, student_id,
        metric_value, metric_unit, performance_text, rating, recorded_by)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
ON CONFLICT (session_id, student_id) DO UPDATE SET
    metric_value=$4, metric_unit=$5, performance_text=$6, rating=$7,
recorded_by=$8
RETURNING *`,
[session_id, program_id, student_id,
metric_value || null, metric_unit || null,
performance_text || null, rating || null, uid(req)]
);
res.status(201).json(result.rows[0]);
} catch (err) {
    res.status(500).json({ message: err.message });
}
});
router.get("/performance/program/:programId", auth, staffOnly, async (req, res)
=> {
    try {
        const result = await pool.query(
            `SELECT u.id AS student_id, u.name, u.username, u.rollno, u.department,
u.batch,
            ts.session_date, p.metric_value, p.metric_unit, p.performance_text,
p.rating
            FROM training_sessions ts
            LEFT JOIN performance p ON p.session_id = ts.id
            LEFT JOIN users u      ON p.student_id = u.id
            WHERE ts.program_id = $1
            ORDER BY u.name, ts.session_date`,

```

```

    [req.params.programId]
  );
  res.json(result.rows);
} catch (err) {
  res.status(500).json({ message: err.message });
}
});

router.delete("/performance/:id", auth, staffOnly, async (req, res) => {
  try {
    await pool.query("DELETE FROM performance WHERE id = $1",
[req.params.id]);
    res.json({ message: "Deleted" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
})

router.get("/selection/program/:programId", auth, staffOnly, async (req, res) =>
{
  const minAtt = parseFloat(req.query.min_attendance ?? 75);
  const minRating = parseFloat(req.query.min_rating ?? 0);
  try {
    const attRows = await pool.query(
      `SELECT u.id AS student_id, u.name, u.username, u.rollno, u.department,
u.batch,
      COUNT(ts.id) AS total_sessions,
      COUNT(a.id) FILTER (WHERE a.present = TRUE) AS
present_count,
      COUNT(a.id) FILTER (WHERE a.present = FALSE) AS
absent_count,

```

```

        ROUND(COUNT(a.id) FILTER (WHERE a.present =
TRUE)::NUMERIC
        / NULLIF(COUNT(ts.id), 0) * 100, 2) AS attendance_percentage,
        BOOL_AND(COALESCE(a.present, FALSE)) AS zero_absence
FROM training_participants tp
JOIN users u ON tp.student_id = u.id
JOIN training_sessions ts ON ts.program_id = tp.program_id
LEFT JOIN attendance a ON a.session_id = ts.id AND a.student_id =
tp.student_id
WHERE tp.program_id = $1
GROUP BY u.id, u.name, u.username, u.rollno, u.department, u.batch`,
[req.params.programId]
);
const perfRows = await pool.query(
`SELECT p.student_id,
        ROUND(AVG(p.rating), 2) AS avg_rating,
        AVG(p.metric_value) AS avg_metric,
        MAX(p.metric_value) AS best_metric,
        MAX(p.metric_unit) AS metric_unit,
        COUNT(p.id) AS perf_sessions
FROM performance p WHERE p.program_id = $1
GROUP BY p.student_id`,
[req.params.programId]
);
const perfMap = {};
perfRows.rows.forEach(p => { perfMap[p.student_id] = p; });
const report = attRows.rows.map(a => {
    const p = perfMap[a.student_id] || {};
    const att = parseFloat(a.attendance_percentage ?? 0);

```

```

const rat = parseFloat(p.avg_rating ?? 0);
const score = parseFloat(((att / 100) * 60 + (rat / 10) * 40).toFixed(2));
return {
  student_id: a.student_id, name: a.name, username: a.username,
  rollno: a.rollno, department: a.department, batch: a.batch,
  total_sessions: parseInt(a.total_sessions),
  present_count: parseInt(a.present_count),
  absent_count: parseInt(a.absent_count),
  attendance_percentage: att,
  zero_absence: a.zero_absence,
  avg_rating: p.avg_rating ? parseFloat(p.avg_rating) : null,
  avg_metric: p.avg_metric ? parseFloat(p.avg_metric) : null,
  best_metric: p.best_metric ? parseFloat(p.best_metric) : null,
  metric_unit: p.metric_unit ?? null,
  selection_score: score,
  recommended: att >= minAtt && (rat === 0 || rat >= minRating)
};
});
report.sort((a, b) => b.selection_score - a.selection_score);
res.json({
  program_id: parseInt(req.params.programId),
  thresholds: { min_attendance: minAtt, min_rating: minRating },
  summary: {
    total_students: report.length,
    recommended_count: report.filter(r => r.recommended).length,
    perfect_attendance_count: report.filter(r => r.zero_absence).length
  },
});
const details = await pool.query(
  `SELECT ts.session_date, ts.start_time, ts.end_time, ts.location,

```

```

        a.present, a.remarks AS att_remarks,
        p.metric_value, p.metric_unit, p.performance_text, p.rating
    FROM training_sessions ts
    LEFT JOIN attendance a ON a.session_id = ts.id AND a.student_id = $2
    LEFT JOIN performance p ON p.session_id = ts.id AND p.student_id = $2
    WHERE ts.program_id = $1
    ORDER BY ts.session_date`,
    [req.params.programId, req.params.studentId]
);
const sessions = details.rows;
const presentCount = sessions.filter(s => s.present === true).length;
const ratings = sessions.filter(s => s.rating).map(s => s.rating);
const avgRating = ratings.length
    ? (ratings.reduce((a, b) => a + b, 0) / ratings.length).toFixed(2) : null;
res.json({
    student: student.rows[0],
    total_sessions: sessions.length,
    present_count: presentCount,
    absent_count: sessions.length - presentCount,
    attendance_percentage: sessions.length
        ? ((presentCount / sessions.length) * 100).toFixed(2) : "0.00",
    avg_rating: avgRating,
    session_details: sessions
});
} catch (err) {
    res.status(500).json({ message: err.message });
}
});
module.exports = router;

```

backend/src/routes/student.routes.js

```
const express = require("express");
const pool = require("../db/db");
const authMiddleware = require("../middleware/auth.middleware");
const roleMiddleware = require("../middleware/role.middleware");
const router = express.Router();
router.get(
  "/events",
  authMiddleware,
  roleMiddleware(["student"]),
  async (req, res) => {
    try {
      const result = await pool.query(
        "SELECT id, title, description, event_type, event_date,
last_registration_date FROM events"
      );
      res.json(result.rows);
    } catch (err) {
      console.error(err.message);
      res.status(500).json({ message: "Error fetching events" });
    }
  });
  const studentId = req.user.id;
  const eventId = req.params.eventId;
  const check = await pool.query(
    "SELECT 1 FROM event_registrations WHERE student_id=$1 AND
event_id=$2",
    [studentId, eventId]);
  if (check.rows.length > 0) {
    return res.status(400).json({ message: "Already registered" });
  }
}
```

```

    }
    await pool.query(
      "INSERT INTO event_registrations (student_id, event_id) VALUES
($1,$2)",
      [studentId, eventId]
    );
    res.json({ message: "Registered successfully" });
  } catch (err) {
    console.error("REGISTER ERROR:", err.message);
    res.status(500).json({ message: "Server error" });
  }
}
);

const result = await pool.query(
  `SELECT
    e.id,
    e.title,
    e.description,
    e.event_type,
    e.event_date,
    e.last_registration_date,
    r.registered_at AS registered_on
  FROM events e
  JOIN event_registrations r ON e.id = r.event_id
  WHERE r.student_id = $1
  ORDER BY r.registered_at DESC`,
  [req.user.id]
);
res.json(result.rows);

```

```

    } catch (err) {
      console.error(err.message);
      res.status(500).json({ message: "Error fetching registered events" });
    }
  }
);

const sessions = await pool.query(
  `SELECT
    ts.id          AS session_id,
    ts.session_date,
    ts.start_time,
    ts.end_time,
    ts.location,
    ts.notes,
    a.present,
    a.remarks      AS att_remarks,
    p.metric_value,
    p.metric_unit,
    p.performance_text,
    p.rating
  FROM training_sessions ts
  LEFT JOIN attendance a ON a.session_id = ts.id AND a.student_id = $2
  LEFT JOIN performance p ON p.session_id = ts.id AND p.student_id =
  $2
  WHERE ts.program_id = $1
  ORDER BY ts.session_date ASC`,
  [programId, studentId]
);

const rows      = sessions.rows;

```



```

    const markedSessions = rows.filter(r => r.present !== null).length;
    const presentCount = rows.filter(r => r.present === true).length;
    const ratings = rows.filter(r => r.rating !== null).map(r =>
parseFloat(r.rating));
    const isParticipant = await pool.query(
        `SELECT 1 FROM training_participants WHERE program_id=$1 AND
student_id=$2`,
        [programId, studentId]
    );
catch (err) {
    console.error("Student training detail:", err.message);
    res.status(500).json({ message: "Error fetching training detail" });
}
});
module.exports = router;

```

backend/src/routes/trainingProgram.js

```

const express = require('express');
const router = express.Router();
const pool = require('../db/db'); // adjust path if needed
router.post('/', async (req, res) => {
    const {
        event_id, title, from_date, to_date,
        location, start_time, end_time, description,
        created_by
    } = req.body;
    if (!event_id || !title || !from_date || !to_date || !start_time || !end_time ||
!created_by) {

```

```

    return res.status(400).json({ error: 'Missing required fields' });
  }
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const prog = await client.query(
      `INSERT INTO training_programs
        (event_id, title, from_date, to_date, location, start_time, end_time,
description, created_by)
        VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9)
        RETURNING *`,
      [event_id, title, from_date, to_date, location, start_time, end_time,
description, created_by]
    );
    const program = prog.rows[0];

    const registered = await client.query(
      `SELECT student_id FROM event_registrations WHERE event_id = $1`,
      [event_id]
    );
    if (registered.rows.length > 0) {
      const values = registered.rows
        .map((r, i) => `($1, $$${i + 2}, $$${registered.rows.length + 2})`)
        .join(', ');
      const params = [program.id, ...registered.rows.map(r => r.student_id),
created_by];
      const insertValues = registered.rows.map((r, i) => `($1, ${r.student_id},
$2)`).join(', ');
      await client.query(

```

```

    `INSERT INTO training_participants (program_id, student_id, added_by)
    VALUES ${registered.rows.map((r) => `($1, ${r.student_id}, $2)`).join(',
')}}
    ON CONFLICT DO NOTHING`,
    [program.id, created_by]
  );
}
const start = new Date(from_date);
const end = new Date(to_date);
const sessionDates = [];
for (let d = new Date(start); d <= end; d.setDate(d.getDate() + 1)) {
  sessionDates.push(new Date(d).toISOString().split('T')[0]);
}
for (const date of sessionDates) {
  await client.query(
    `INSERT INTO training_sessions (program_id, session_date, start_time,
end_time, location)
    VALUES ($1, $2, $3, $4, $5)`,
    [program.id, date, start_time, end_time, location]
  );
}
await client.query('COMMIT');
res.status(201).json({
  message: 'Training program created with sessions and participants',
  program,
  sessions_generated: sessionDates.length,
  participants_added: registered.rows.length
});
} catch (err) {

```

```

    await client.query('ROLLBACK');
    console.error(err);
    res.status(500).json({ error: err.message });
  } finally {
    client.release();
  }
});

router.get('/', async (req, res) => {
  const { event_id } = req.query;
  try {
    const query = `
      SELECT tp.*, e.title AS event_title, u.username AS created_by_name,
        COUNT(DISTINCT tpa.student_id) AS participant_count,
        COUNT(DISTINCT ts.id) AS session_count
      FROM training_programs tp
      JOIN events e ON tp.event_id = e.id
      JOIN users u ON tp.created_by = u.id
      LEFT JOIN training_participants tpa ON tpa.program_id = tp.id
      LEFT JOIN training_sessions ts ON ts.program_id = tp.id
      ${event_id ? 'WHERE tp.event_id = $1' : ''}
      GROUP BY tp.id, e.title, u.username
      ORDER BY tp.from_date DESC
    `;
    const result = await pool.query(query, event_id ? [event_id] : []);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

router.get('/:id', async (req, res) => {
  try {
    const prog = await pool.query(
      `SELECT tp.*, e.title AS event_title, u.username AS created_by_name
      FROM training_programs tp
      JOIN events e ON tp.event_id = e.id
      JOIN users u ON tp.created_by = u.id
      WHERE tp.id = $1`,
      [req.params.id]
    );
    if (!prog.rows.length) return res.status(404).json({ error: 'Not found' });
    const sessions = await pool.query(
      `SELECT * FROM training_sessions WHERE program_id = $1 ORDER
BY session_date`,
      [req.params.id]
    );
    const participants = await pool.query(
      `SELECT tp.student_id, u.username, u.department, u.batch
      FROM training_participants tp
      JOIN users u ON tp.student_id = u.id
      WHERE tp.program_id = $1
      ORDER BY u.username`,
      [req.params.id]
    );
    res.json({
      program: prog.rows[0],
      sessions: sessions.rows,
      participants: participants.rows
    });
  }
}

```

```

    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

router.put('/:id', async (req, res) => {
  const { title, from_date, to_date, location, start_time, end_time, description } =
    req.body;

  try {
    const result = await pool.query(
      `UPDATE training_programs
      SET title=$1, from_date=$2, to_date=$3, location=$4,
        start_time=$5, end_time=$6, description=$7, updated_at=NOW()
      WHERE id=$8 RETURNING *`,
      [title, from_date, to_date, location, start_time, end_time, description,
req.params.id]
    );

    if (!result.rows.length) return res.status(404).json({ error: 'Not found' });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

router.delete('/:id', async (req, res) => {
  try {
    await pool.query('DELETE FROM training_programs WHERE id = $1',
[req.params.id]);

    res.json({ message: 'Training program deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

    }
  });
  router.post('/:id/participants', async (req, res) => {
    const { student_id, added_by } = req.body;
    try {
      const result = await pool.query(
        `INSERT INTO training_participants (program_id, student_id, added_by)
        VALUES ($1, $2, $3) ON CONFLICT DO NOTHING RETURNING *`,
        [req.params.id, student_id, added_by]
      );
      res.status(201).json(result.rows[0] || { message: 'Already exists' });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });
  router.delete('/:id/participants/:studentId', async (req, res) => {
    try {
      await pool.query(
        `DELETE FROM training_participants WHERE program_id=$1 AND
        student_id=$2`,
        [req.params.id, req.params.studentId]
      );
      res.json({ message: 'Participant removed' });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });
  router.post('/:id/sessions', async (req, res) => {
    const { session_date, start_time, end_time, location, notes } = req.body;

```

```

try {
  const result = await pool.query(
    `INSERT INTO training_sessions (program_id, session_date, start_time,
end_time, location, notes)
    VALUES ($1,$2,$3,$4,$5,$6) RETURNING *`,
    [req.params.id, session_date, start_time, end_time, location, notes]
  );
  res.status(201).json(result.rows[0]);
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

router.put('/:id/sessions/:sessionId', async (req, res) => {
  const { session_date, start_time, end_time, location, notes } = req.body;
  try {
    const result = await pool.query(
      `UPDATE training_sessions
      SET session_date=$1, start_time=$2, end_time=$3, location=$4, notes=$5
      WHERE id=$6 AND program_id=$7 RETURNING *`,
      [session_date, start_time, end_time, location, notes, req.params.sessionId,
req.params.id]
    );
    if (!result.rows.length) return res.status(404).json({ error: 'Not found' });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

router.delete('/:id/sessions/:sessionId', async (req, res) => {

```



```
try {  
  await pool.query('DELETE FROM training_sessions WHERE id=$1 AND  
program_id=$2',  
    [req.params.sessionId, req.params.id]);  
  res.json({ message: 'Session deleted' });  
} catch (err) {  
  res.status(500).json({ error: err.message });  
}  
});  
module.exports = router;
```